SISTEMA WEB PARA LA GESTIÓN DE ALQUILER DE VEHÍCULOS

Grupo 03

Docente: Jose Luis Milla Flores

INTEGRANTES

Stefano T.
Bravo Rosales

Yvan Y. Sodani
Almonacid

Marjhorick A.
Ventura Peña

Realidad Problemática

Actualmente, muchas empresas de alquiler de vehículos gestionan sus operaciones de manera manual o mediante herramientas poco integradas, lo que genera problemas como:

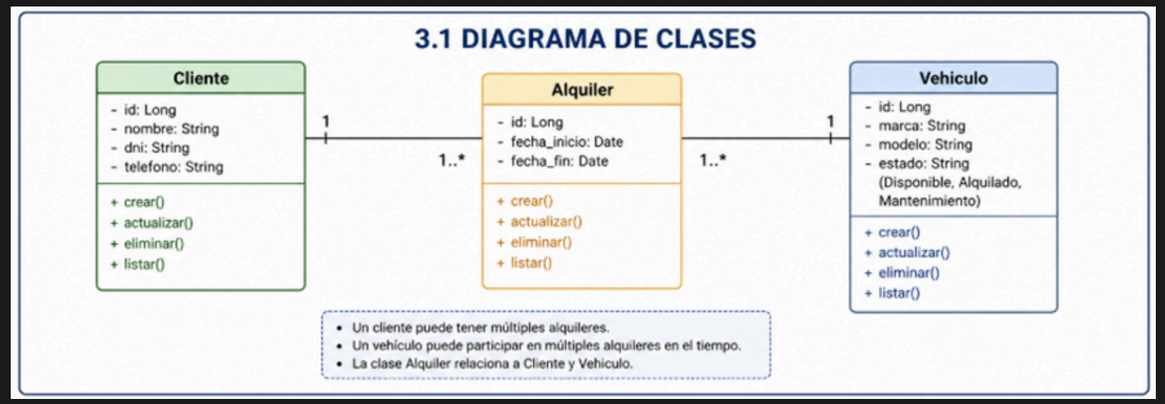
- Falta de control en la disponibilidad de vehículos
- Errores en el registro de alquileres
- Dificultad en el seguimiento de clientes
- Desorganización de la información

Objetivos generales

Desarrollar un sistema web para la gestión de alquiler de vehículos que permita administrar eficientemente la información de clientes, vehículos y alquileres.

DIAGRAMA DE CLASES

- **Cliente:** almacena información del cliente (id, nombre, dni, teléfono).
- **Vehículo:** almacena datos del vehículo (id, marca, modelo, estado).
- **Alquiler:** representa la relación entre cliente y vehículo (id, fecha_inicio, fecha_fin).



MODELO ENTIDAD – RELACION

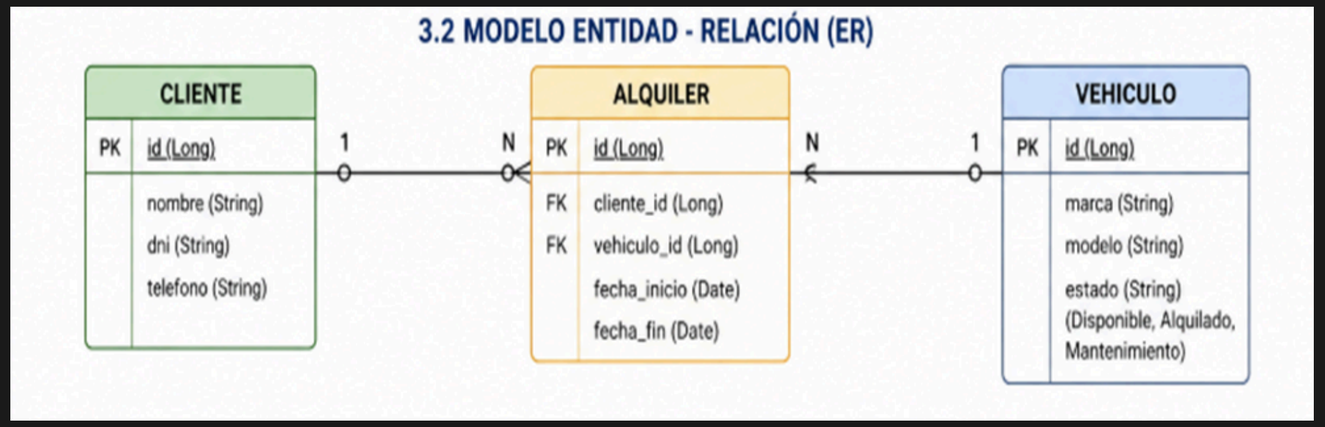
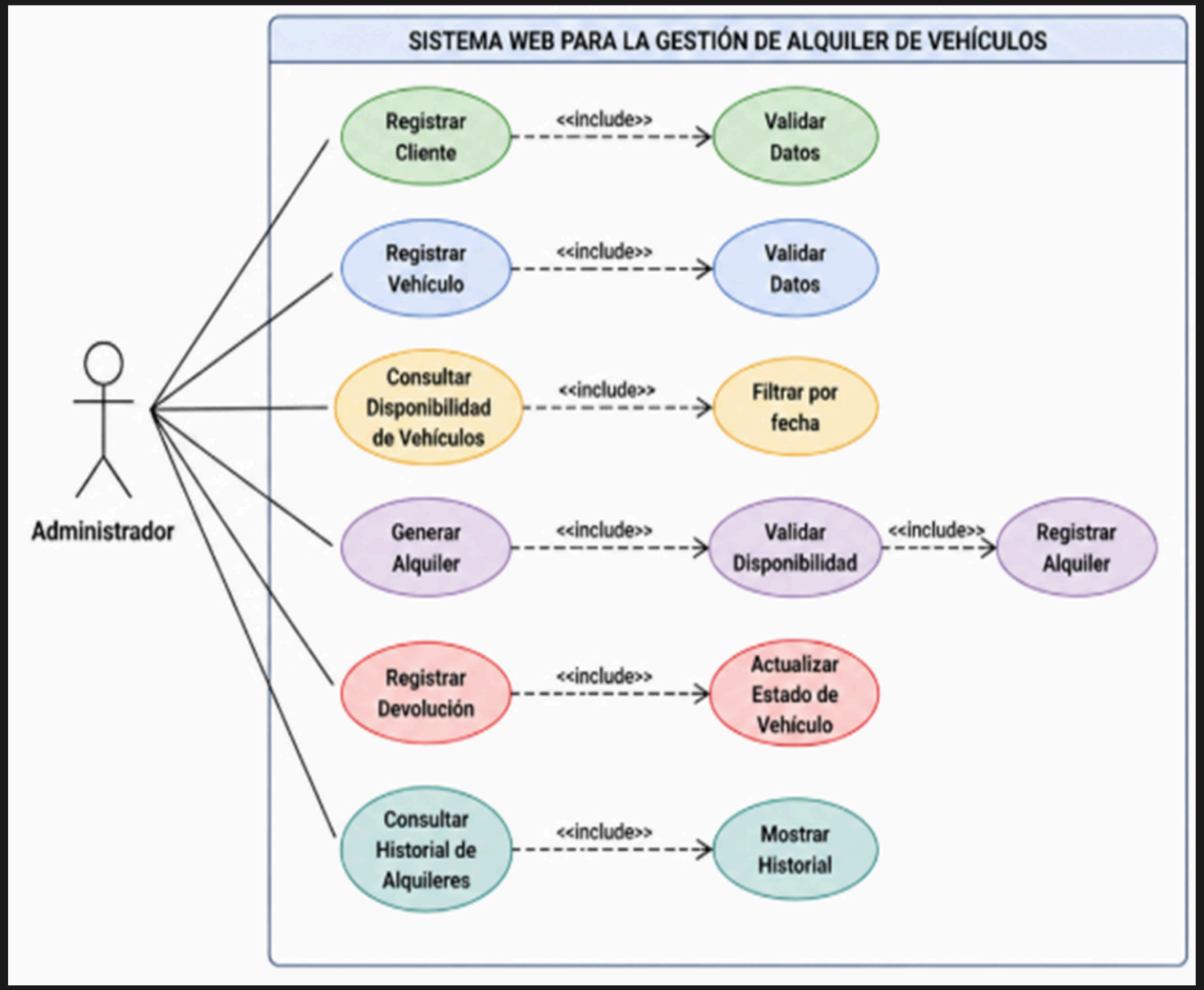


DIAGRAMA DE CASOS DE USO



MARCO TEÓRICO

Control de Versiones con Git

Permite mantener un control ordenado del código y evitar conflictos entre integrantes.

GitHub

Plataforma que permite alojar repositirio Git en la nube

Arquitectura REST

Esta arquitectura facilita la escalabilidad del sistema y permite que en el futuro se pueda integrar con aplicaciones móviles o frontend web

Persistencia de datos con JPA/Hibernate

Esto permite realizar operaciones CRUD sin necesidad de escribir consultas SQL complejas.

Base de datos relacional en el sistema

Esto permite mantener la integridad de los datos y facilitar consultas como historial de alquileres o disponibilidad de vehículos.

Spring Boot en la implementación del backend

Spring Boot facilita la creación rápida de APIs REST para gestionar clientes, vehículos y alquileres.

METODOLOGÍA

Scrum

- Se dividen las funcionalidades
- Cada integrante es responsable de una parte del sistema.
- Se realiza seguimiento del avance mediante reuniones o coordinación del equipo.

Git Flow

Organizacion del proyecto

- **Rama main:** contiene la versión final estable del sistema.
- **Rama develop:** contiene la integración de todas las funcionalidades en desarrollo.
- **Ramas feature:** se crean para cada funcionalidad específica.

Flujo de trabajo con Pull Requests

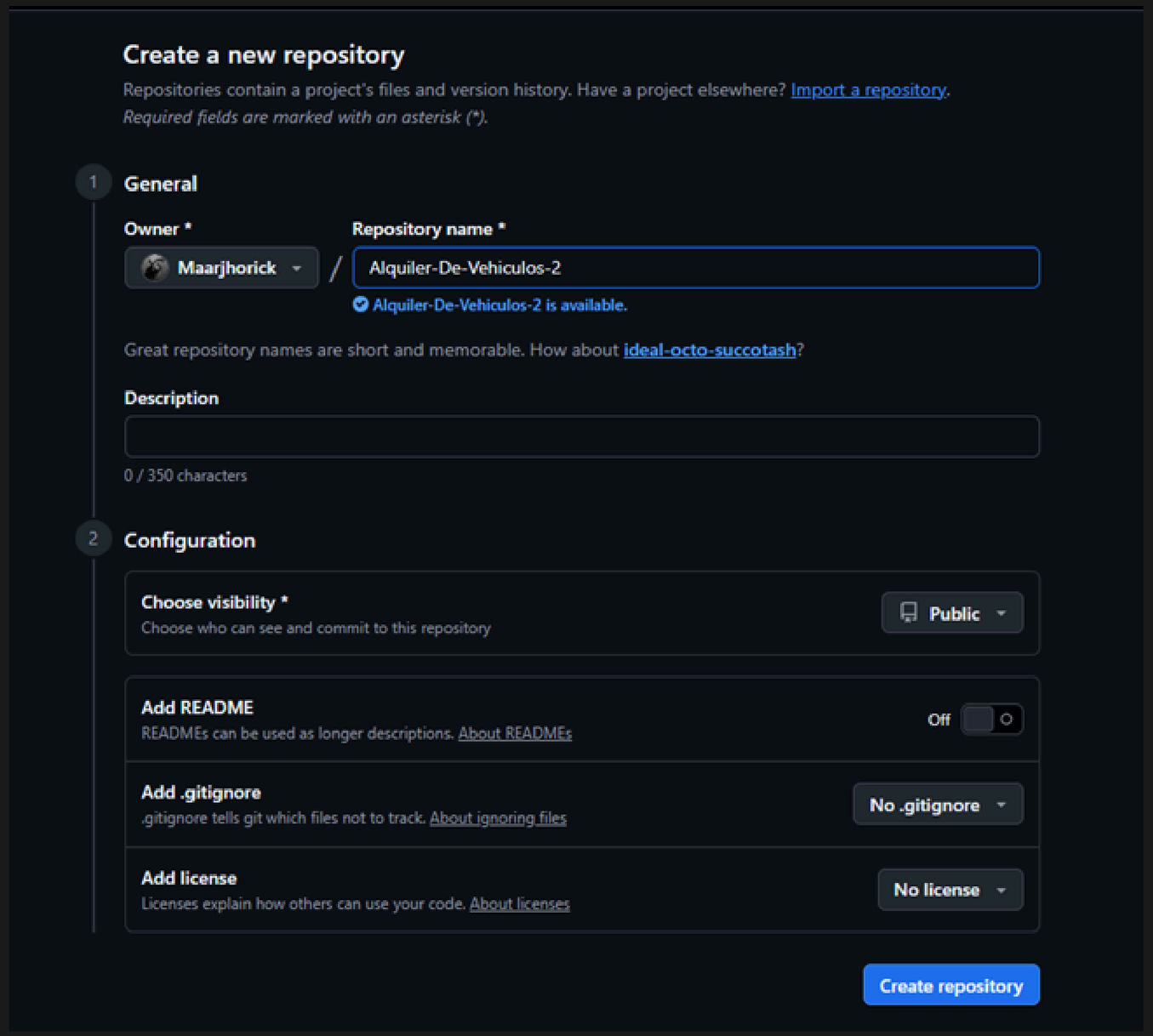
- El desarrollador crea una rama feature
- Implementa la funcionalidad
- Realiza commits usando Conventional Commits
- Sube los cambios al repositorio remoto
- Crea un Pull Request hacia la rama develop
- Otro integrante revisa el código
- Si es aprobado, se realiza el merge.

Aplicación de Conventional Commits

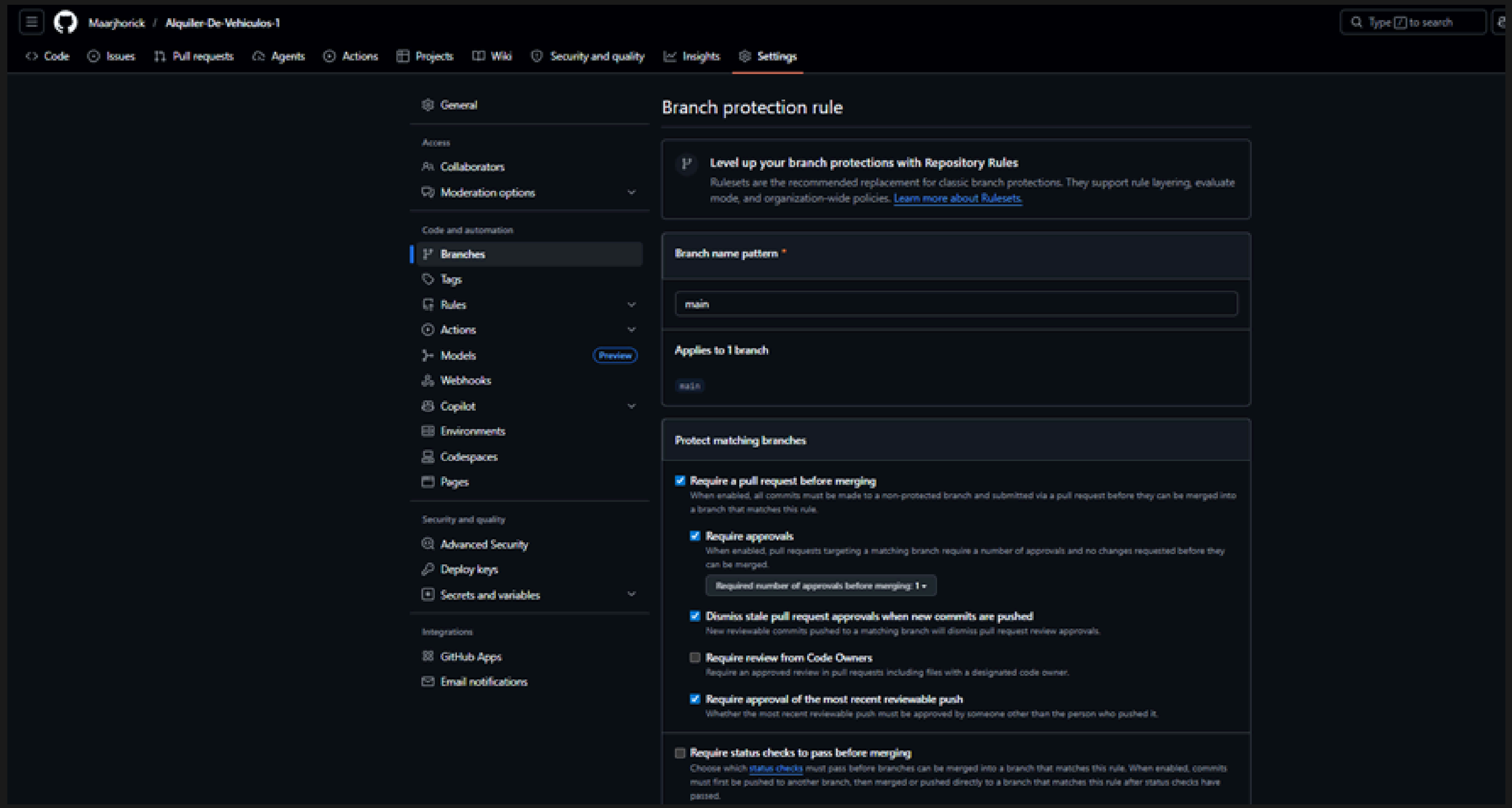
Prefijos en los commits

- **feat:** cuando se agrega una nueva funcionalidad
- **fix:** cuando se corrige un error
- **docs:** cuando se modifica documentaci

Crear un nuevo repositorio



Protección de la rama main



Pull Request

Configuracion a la base de datos #11

Open Yamil-Sodani wants to merge 1 commit into `main` from `Yamil-Sodani-patch-2`

Conversation 0 | Commits 1 | Checks 0 | Files changed 1

Yamil-Sodani commented 2 minutes ago — Collaborator

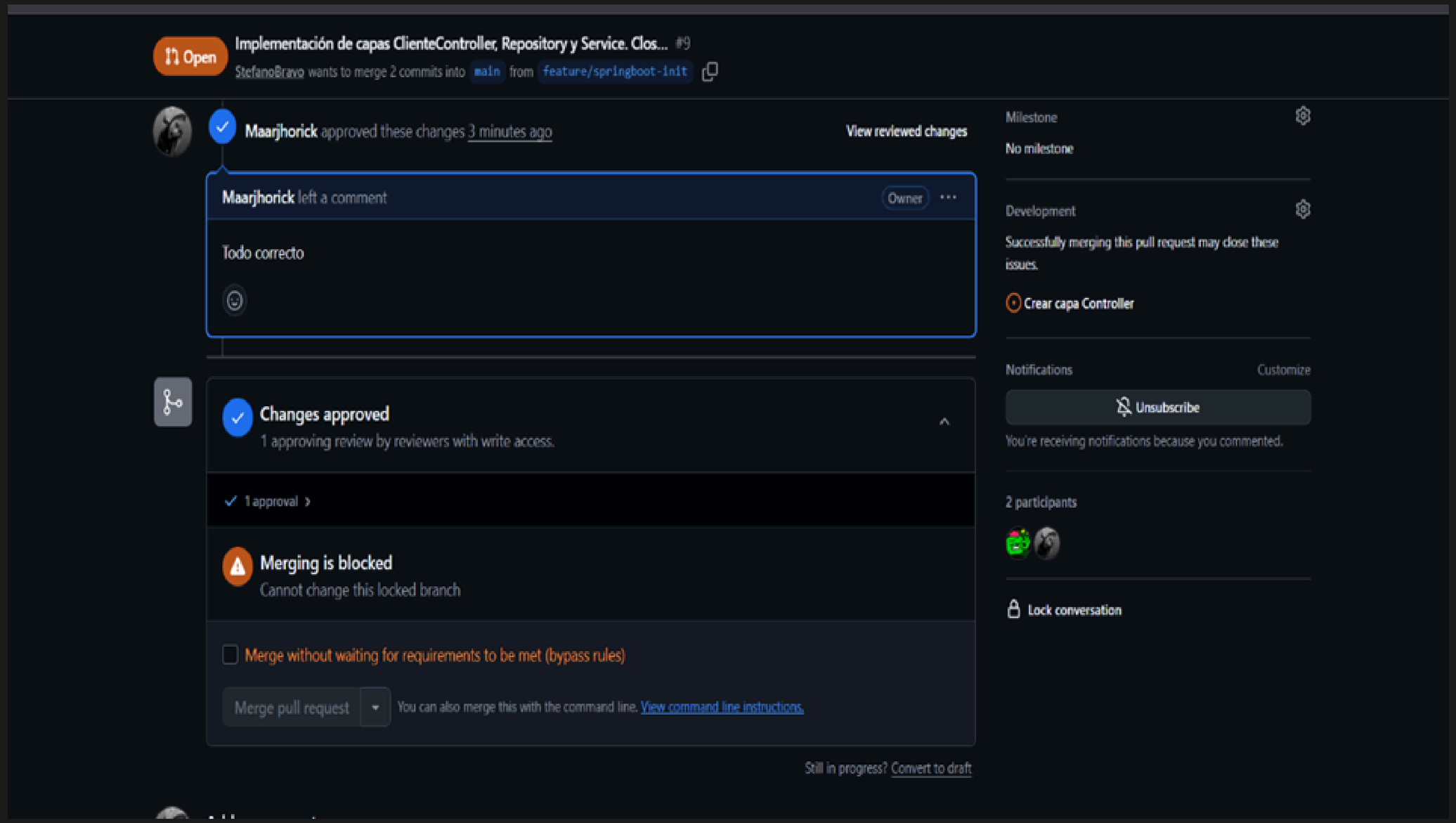
No description provided.

`Configuracion a la base de datos` — Verified — 27e7387

Review required
At least 1 approving review is required by reviewers with write access.

Merging is blocked
New changes require approval from someone other than the last pusher.
Cannot change this locked branch

Pull Request aprobado



Merge realizado

Configuracion a la base de datos #11

Merged Maarjhorick merged 1 commit into `main` from `Yamil-Sodani-patch-2` now

Conversation 0 | Commits 1 | Checks 0 | Files changed 1

Yamil-Sodani commented 5 minutes ago — Collaborator

No description provided.

`Configuracion a la base de datos` — Verified — 27e7387

Maarjhorick approved these changes 1 minute ago — View reviewed changes

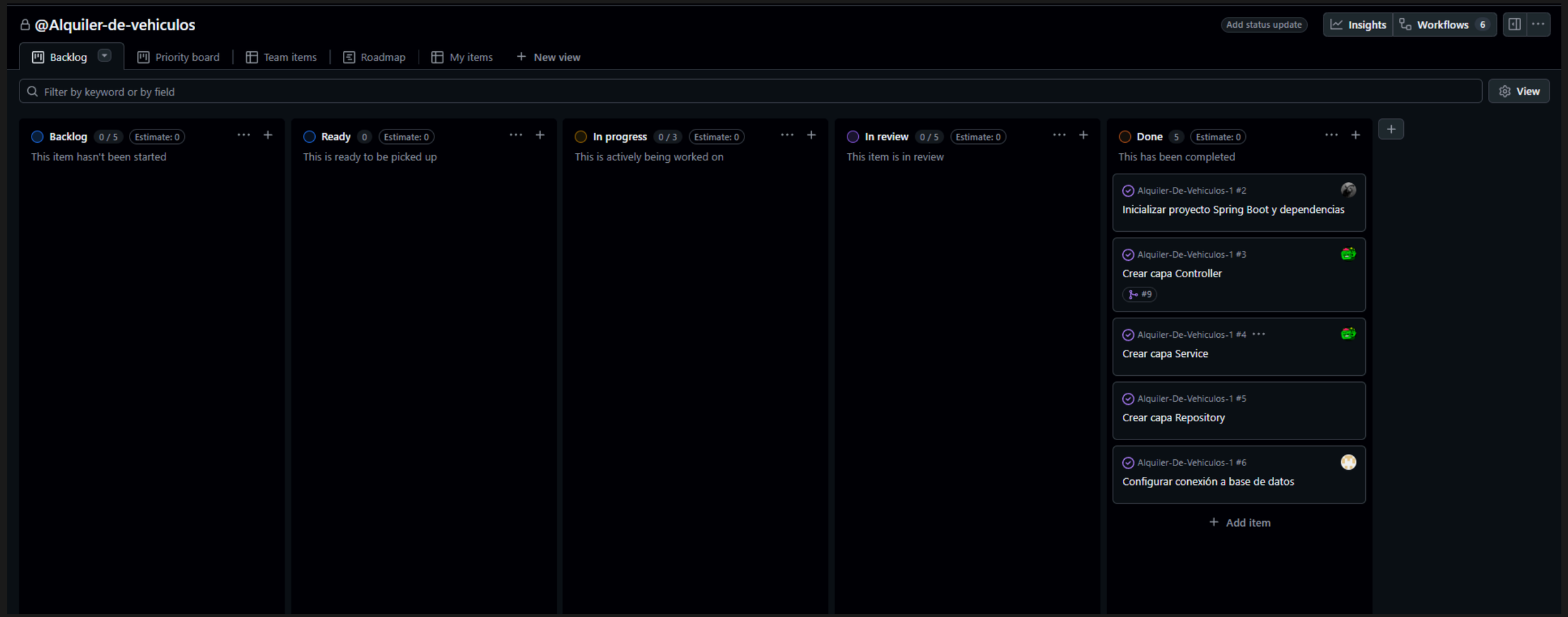
Maarjhorick merged commit **c2e4d72** into `main` now — Revert

Pull request successfully merged and closed

You're all set — the `Yamil-Sodani-patch-2` branch can be safely deleted.

Delete branch

Tablero Kanban



MUCHAS GRACIAS !